

# Better `std::tuple` Indexing

Document #: P2726R0  
Date: 2022-11-18  
Project: Programming Language C++  
Audience: LEWG-I  
LEWG  
Reply-to: Zach Laine  
<[whatwasthataddress@gmail.com](mailto:whatwasthataddress@gmail.com)>

## § 1 The ergonomics of `std::tuple<int>` can be improved

A common complaint that I hear about `std::tuple` from C++ users is that getting the Nth value out of it is painfully verbose. Fortunately, if [P2725 `std::integral\_constant` literals](#) is adopted, we can do better.

| Before                                                                                                                                                                                                 | After                                                                                                                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>auto t = std::tuple&lt;int,<br/>    std::string&gt;{42,<br/>    "how many ..."};<br/>assert(std::get&lt;0&gt;(t) ==<br/>    42);<br/>assert(std::get&lt;1&gt;(t) ==<br/>    "how many...");</pre> | <pre>auto t = std::tuple&lt;int, std::string&gt;<br/>    {42, "how many ..."};<br/>using namespace std::literals;<br/>assert(t[0ic] == 42);<br/>    // Option 1.<br/>assert(std::get(t, 1ic) == "how<br/>    many..."); // Option 2.</pre> |

The expressions `std::get<0>(t)` and `t[0ic]` are semantically identical, but syntactically the former is very noisy and the latter is not. All either operation does is get a reference to the Nth element of the tuple. `t[0ic]` expresses that concisely.

## § 2 Proposed Design

There are multiple options here, as indicated in the initial example above. Please note that both the options rely on the existence of [P2725](#).

### § 2.1 Option 1, A.K.A. “The Author’s Favorite”

Add an operator[] to `std::tuple`.

This design does not come from me. It is the way that [Boost.Hana](#)’s tuples work. It’s been around a long time, and people really seem to like it.

In more indexing-heavy code, Boost.Hana-style concision really helps. Say you have a context object `ctx` that contains a large number of tuples used to capture configury and transient state, and multiple accessors `_foo()` that return references to tuples in `ctx`:

| Before                                                                                                                                                                                                                                                                                                                                   | After                                                                                                                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>if (enable_caching) {     std::get&lt;0&gt;(_locals(ctx)) =         std::get&lt;0&gt;(_attrs(ctx));     std::get&lt;1&gt;(_locals(ctx)) =         std::get&lt;1&gt;(_attrs(ctx)); } _val(ctx) = make_result(     std::move(std::get&lt;0&gt;         (_attrs(ctx))),     std::move(std::get&lt;1&gt;         (_attrs(ctx))));</pre> | <pre>if (enable_caching) {     _locals(ctx)[0ic] =         _attrs(ctx)[0ic];     _locals(ctx)[1ic] =         _attrs(ctx)[1ic]; } _val(ctx) = make_result(     std::move(_attrs(ctx)         [0ic]),     std::move(_attrs(ctx)         [1ic]));</pre> |

Also, in any situation where tuples are nested, the use of the index operator makes things much clearer:

| Before                                                   | After                        |
|----------------------------------------------------------|------------------------------|
| <pre>std::get&lt;2&gt;(std::get&lt;1&gt;(t)) = 42;</pre> | <pre>t[1ic][2ic] = 42;</pre> |

## § 2.2 Option 2, A.K.A. “The ‘Meh’ One”

There is an alternative to adding a new operation to `std::tuple` – we could just add an overload of `std::get()` that takes a `std::integral_constant` as a function parameter. I don’t think the results are nearly as nice:

| Before                                                                                                                                                                                                                                                                                                                                                             | After                                                                                                                                                                                                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>if (enable_caching) {     std::get&lt;0&gt;(_locals(ctx))         = std::get&lt;0&gt;             (_attrs(ctx));     std::get&lt;1&gt;(_locals(ctx))         = std::get&lt;1&gt;             (_attrs(ctx)); } _val(ctx) = make_result(     std::move(std::get&lt;0&gt;         (_attrs(ctx))),     std::move(std::get&lt;1&gt;         (_attrs(ctx))));</pre> | <pre>if (enable_caching) {     std::get(_locals(ctx), 0ic) =         std::get(_attrs(ctx), 0ic);     std::get(_locals(ctx), 1ic) =         std::get(_attrs(ctx), 1ic); } _val(ctx) = make_result(     std::move(std::get(_attrs(ctx),         0ic)),     std::move(std::get(_attrs(ctx),         1ic)));</pre> |

This effectively replaces `<>` with `,` and `ic`, which is slightly more typing. It also leaves the noisiest part, `std::get`, still in play.

This option does have the advantage that it could be used to address non-`std::tuple` uses of `std::get()` as well (though that is not proposed here). If you happen already to have a `std::integral_constant ic` lying about, you can use it directly as a function call arg. It saves you from having to type `ic.value`, I guess.

This option helps slightly in a nested-tuple situation, in that the indices no longer appear in reverse order:

| Before                                                       | After                                                |
|--------------------------------------------------------------|------------------------------------------------------|
| <pre>std::get&lt;2&gt;(std::get&lt;1&gt;(t)) =<br/>42;</pre> | <pre>std::get(std::get(t, 1ic), 2ic) =<br/>42;</pre> |

## § 3 Implementation experience

As stated earlier, this has been implemented in Boost.Hana's tuple for years. The implementation is straightforward, especially since all we need to do is add a new operator[] that just calls `std::get()`, (Option 1) or add a new set of overloads of `std::get()` each of which calls one of the old ones (Option 2).

## § 4 Option 1 Wording

In 22.4.4 [tuple.tuple], add this new member function to tuple:

```
template<class Self, class IndexType, IndexType I>  
constexpr decltype(auto) operator[](this Self && self,  
    integral_constant<IndexType, I>  
    { return std::get<I>(std::forward<Self>(self)); }
```

## § 5 Option 2 Wording

In 22.4.2 [tuple.syn], append these function templates to the end of the [tuple.elem] section:

```
template<class IndexType, IndexType I, class... Types>  
constexpr decltype(auto) get(tuple<Types...>& t) noexcept { return  
    std::get<I>(t); }  
template<class IndexType, IndexType I, class... Types>  
constexpr decltype(auto) get(tuple<Types...>&& t) noexcept { return  
    std::get<I>(std::move(t)); }  
template<class IndexType, IndexType I, class... Types>  
constexpr decltype(auto) get(const tuple<Types...>& t) noexcept {  
    return std::get<I>(t); }
```

```
template<class IndexType, IndexType I, class... Types>
constexpr decltype(auto) get(const tuple<Types...>&& t) noexcept {
    return std::get<I>(std::move(t)); }
```